ANS Forth Test Suite

Introduction

The ANS Forth Test Suite is intended to prove that a forth distribution
is in compliance with the ANS Forth standard.
However full compliance requires other conditions to be satisfied.
Those conditions are not satisfied in this project.

> Source: https://forth-standard.org/standard/testsuite (Annex F), fetched 2026-07-15. Explicitly redistributable per the test harness's own header: "(C) 1995 JOHNS HOPKINS UNIVERSITY / APPLIED PHYSICS LABORATORY - MAY BE DISTRIBUTED FREELY AS LONG AS THIS COPYRIGHT NOTICE REMAINS."

The test suite files here have been modified
to facilitate development testing, instead of compliance testing.
This includes development by manual cut/paste of forth source into a terminal emulator
or by running the supplied automation script.

Modifications include:

1. File splits by section.
2. Sections can be executed individually, without dependencies on prior tests.
3. Performance enhancements.
4. Improved reporting (`TEST-BEGIN` / `TEST-END` , see *Test Reporting* below).
5. Corrections for test expectations that violate the standard.
6. Adaptations for a 16-bit cell.

Testing can target serial-connected hardware (MECB 6809)
or the MAME software emulation of a 6809 SBC with a 6850 ACIA.

Running the Suite

MAME Emulator Setup

The MAME emulator needs to be setup in advance.
Refer to the instructions in

```
${HOME}/git/ClaudeForth/MECB6809\ Emulation.
```

This includes, from one terminal:

1. Replacing the buggy 6850acia.cpp file
   provided in the MAME git project with the modified version
   provided in the ClaudeForth git project.
2. Installing the mecb6809.cpp driver source file.
3. Building the MAME project.
4. Assembling then installing, the forth6809.asm
   as a .bin file into the MAME rom directory.

```
# The update to the MAME executable.
cp \
    ${HOME}/git/ClaudeForth/MECB6809\ Emulation/6850acia.cpp \
    ${HOME}/git/mame0288/src/devices/machine/6850acia.cpp
cp \
    ${HOME}/git/ClaudeForth/MECB6809\ Emulation/mecb6809.cpp \
    ${HOME}/git/mame0288/src/mame/homebrew/mecb6809.cpp
cd ${HOME}/git/mame0288
make SUBTARGET=mecb6809 SOURCES=src/mame/homebrew/mecb6809.cpp TOOLS=1 REGENIE=1
```

Then in another terminal:

```
# Providing the ROM file.
cd ${HOME}/git/ClaudeForth
lwasm --6809 --format=raw \
   --output=forth6809.bin --list=forth6809.lst \
   --define=UNITTESTS=0 --define=TSTSELECTOR=15 \
   --define=SERIALPOLL=1 \
   forth6809.asm
mkdir "${HOME}/Library/Application Support/mame/roms/mecb6809"
cp forth6809.bin "${HOME}/Library/Application Support/mame/roms/mecb6809/mecb6809
```

Python Test Runner

Usage

In a further terminal,
assuming a git clone to download this project,
navigate to the top level ClaudeForth directory
and then to the ANS Test Suite. Execute the test runner script:

```
# Go there
cd ${HOME}/git/ClaudeForth/ANS\ Test\ Suite
# Verify
ls -al
# Execute the tests suite
python3 ans_test_runner.py \
    --target mame \
    --mame-bin "$HOME/git/mame0288/mecb6809" \
    --retrigger-cmd $HOME/git/ClaudeForth/ANS\ Test\ Suite/retrigger.cmd \
    --skip-assemble \
    --ans-dir $HOME/git/ClaudeForth/ANS\ Test\ Suite/ans_tests \
    --sections 08 09 10 11 12 13 14 15 16 17 18 19 20 21 23 24 26 27 28 29 30 31
    --char-delay 0.05 \
    --retries 0 \
    --log-dir $HOME/git/ClaudeForth/ANS\ Test\ Suite/ans_test_results
```

`ans_test_runner.py` automates loading the setup forth files
into a running forth6809 image.
Then it loads the forth tests.
The test runner reports a summary PASS/FAIL
per section into the stderr of the terminal.
The full transcript of each section,
including the `TEST SUMMARY` and any ttester assertion failure messages,
are written to a per-section log file in the `--log-dir` directory.

A section PASSes when its `TEST SUMMARY` line reports zero failures
and the section loaded without a compile error or line timeout.
A section with no `TEST SUMMARY` line at all FAILs.
The runner prints a warning if it sees stack imbalance notes.

To locate the data stack cell at which a section leaks,
add the `--trace-depth` option.
The runner then asks the target for `DEPTH` after every line it sends.

Operating Principles

The python test runner talks to either a MAME instance
(spawned the same way `run_all_tests.sh` does,
over a `-rs232 null_modem -bitb socket.host:port` TCP connection)
or a real serial-connected MECB6809 with the Forth
binary in EPROM (via pyserial).

The test runner applies per character delays, but doesn't apply end-of-line delays.
Instead it uses forth6809's *own* line-by-line `ok` / `ERROR` reply
as the pacing signal.
In other words: one line goes out,
the runner blocks until that line's own reply comes back,
then the next line goes out.
Long test-file lines (some exceed 1000 characters) are automatically
re-wrapped to fit under `TIBBUFL` (80 bytes)
without ever splitting inside a `S"` / `."` / `\` construct
or inside a `T{ ... }T` assertion.
Software (XON/XOFF) flow control from the target is honoured.

The runner also removes the dead `HAS-FLOATING` branches of `ttester.fs`
before sending it (the target has no floating-point word set),
which removes most of the harness load time.

See the script's own `--help` and
module docstring for every option.

Manual Cut and Paste Via Terminal Emulator

Usage

For full communication details refer to the file:

```
more ${HOME}/git/ClaudeForth/MAME_to_minicom.md
```

In yet another terminal start up the socat bridge:

```
socat PTY,link=$HOME/mame-pty,raw,echo=0,ixon=1,ixoff=1 TCP-LISTEN:11185,reuseadd
```

This must be started first.

In yet another terminal startup minicom:

```
minicom 57600_XONXOFF
```

Note: refer to MAME_to_minicom.md for the
location and construction of the
~/.minirc.57600_XONXOFF stored settings file.

In yet another terminal startup MAME as the MECB 6809 emulator:

```
cd $HOME/git/mame0288
./mecb6809 mecb6809 \
   -rs232 null_modem -bitb socket.127.0.0.1:11185  \
   -throttle  \
   -window -resolution 640x480 \
   -debug
```

The terminal emulator should now respond to keyboard input,
with the per character delay and handshaking preset.

In a text editor supporting copy and paste, choose a forth source code file,
select the text, copy it to the clipboard and then paste it into the minicom window.
All the files are loaded in this way,
in the same order as used by the python test runner:

1. `ttester.fs`
2. `00_test_prelude.fs`
3. any section files, in any order.

Each section file prints its own `TEST SUMMARY` at its end,
so no runner is needed to read the result.
Do not paste a file that has been reflowed by an editor:
see *Editing Test Files* below.

Test Reporting

Each section file reports for itself, using two words
defined in `00_test_prelude.fs` :

Word	Purpose
`TEST-BEGIN`	Zeroes the pass and fail counters (`TESTCOUNT` , `FAILCOUNT` , incremented by `}T` in `ttester.fs`) and records the current data stack depth.
`TEST-END`	Prints the file's statistics, then checks the data stack depth against the depth recorded by `TEST-BEGIN` . Always prints in DECIMAL and preserves `BASE` .

Every section file calls `TEST-BEGIN` on the line after its `MARKER Mnn`
and `TEST-END` on the line before its closing `Mnn` .
The prelude calls them around its own checks as well.

Typical output:

```
TEST SUMMARY: 125 run, 0 failed
```

If a file leaves the data stack deeper than it found it:

```
TEST SUMMARY: 31 run, 0 failed
STACK IMBALANCE: 3 extra cell(s) dropped
```

The extra cells are dropped so the next file starts clean.
A shallower stack is reported as `n cell(s) missing`.

The stack check matters because `T{ ... }T` measures depth relative to its own starting depth, so a stray cell left by an earlier test is invisible to it.
A file can show zero failures and still leak cells.

When a test fails, ttester prints a message in the terminal such as `INCORRECT RESULT:` followed by the test source line.
A failure counted in `TEST SUMMARY` always has such a message earlier in the output.

Editing Test Files

A few tests depend on the physical line layout of the file,
because the target's `ACCEPT` gives each test exactly one input line.
These must stay split exactly as they are:

- `19_string_words.tests.fs`: `T{ BL GS3 / DROP -> 0 }T`, and both `T{ PARSE-NAME / NIP -> 0 }T` pairs. They need an empty parse area, i.e. the end of the line must be reached.
- `24_environmental_query.tests.fs`: the `>IN` tests, the three-line `RESCAN?` test, and `GS4`.

Other guidance:

- Keep one test per line, so a failure message identifies it.
- Never put a test on the same physical line as a `\` comment. The rest of the line is swallowed and the test silently never runs.
- A bare `DECIMAL` or `HEX` leaks into the next section file, because `MARKER` restores the dictionary but not `BASE`. Restore it unconditionally, as the existing files do.
- `>R` / `R>` must not span separate top-level lines. Better still, keep them inside a definition: their interpretation semantics are undefined, and at the top level they leave the value under the interpreter's own return address (a `T{ ... >R ... R> ... }T` line in `29_coreext_strings.tests.fs` reset the

target until it was wrapped in a colon definition).

- Keep `TEST-BEGIN` after `MARKER Mnn` and `TEST-END` before the closing `Mnn`. For a new section file that already has its `MARKER Mnn` lines, `python3 apply_test_markers.py ans_tests [--dry-run]` inserts both. It examines every file in the directory, skips files that already have them, and keeps no backup, so use `--dry-run` first or run it under version control.

Manifest

1. Python test runner to automate testing.
2. Support files used by the python test runner.
3. Forth setup files that are loaded into the test target prior to running tests. These are located in the sub-directory ans_tests.
4. The ans_tests sub-directory containing the forth test files organised according to the glossary sections for forth6809.
5. The ans_test_results sub-directory containing the log files recording previous test run results.
6. Miscellaneous utility scripts.

What follows is a more detailed description of the contents of ans_tests and the function of each file.

Forth Conditional Compilation Support

`[IF]`, `[ELSE]` and `[THEN]` from the Programming-Tools word set are part of the ROM since stage 6d, so `ttester.fs` (which gates its floating point code behind `HAS-FLOATING [IF]`) loads directly. The former utility file `00a_tool_ext_conditionals.fs` is no longer needed and should be removed from the harness list in `ans_test_runner.py`, the line that reads `for fname in ("00a_tool_ext_conditionals.fs", "ttester.fs", "00_test_prelude.fs"):`.

`ttester.fs` needs them immediately, in its own conditional compilation of the `HAS-FLOATING` / `HAS-FLOATING-STACK` source blocks, so this file loads first, ahead of even `ttester.fs`.

`[IF]` / `[ELSE]` / `[THEN]` is provided as a reference implementation by the standard itself (forth-standard.org/standard/tools/BracketELSE), built only from words forth6809.asm already has (`BL WORD COUNT S" COMPARE REFILL IF/ELSE/THEN BEGIN/WHILE/REPEAT/UNTIL ?DUP EXIT IMMEDIATE`) — nothing further needs

to be assembled into the ROM to
run the ANS suite.

ANS Forth Test Harness

`ttester.fs` — represents the test harness itself
(ANS Forth Standard, Annex F, Section F.2.3).

It defines the test setup/assertion words `T{` / `->` / `}T`
and every supporting word, including:

1. `ERROR`
2. `EMPTY-STACK`
3. `HAS-FLOATING` / `HAS-FLOATING-STACK` detection.
4. `X}T` / `R}T` assertion words for mixed cell/float stack pictures.

Load this file first, before the prelude or any section file.

The extraction of this file was validated by checking nesting of:

1. `[IF]` and `[THEN]` . Counts match exactly (8/8)
2. `:` / `;` counts match exactly (63/63).

No unbalanced conditional-compilation or colon
definitions were introduced while extracting it from the original source page.

ANS Forth Test Prelude

The prelude file `00_test_prelude.fs` is loaded after `ttester.fs` ,
but before the test section files.
It provides shared section setup utilities with support for:

1. Test reporting (`TEST-BEGIN` , `TEST-END`).
2. Two's-complement assumptions (BITSSET?).
3. Bound constants (MAX-UINT, MAX-INT, MIN-INT, MID-UINT, MID-UINT+1, MSB).
4. Boolean constants (0S/1S via <FALSE>/<TRUE>).
5. Floored-vs-symmetric division helpers (IFFLOORED/IFSYM).
6. String-compare helper (S=).
7. Shared memory buffers used by the FILL/MOVE tests (FBUF/SBUF/SEEBUF).

ANS Forth Tests

The ANS Forth test files, `NN_name.tests.fs` ,

are aligned with one file per forth6809.asm section
(matching the exact same numbering and filenames as `forth6809_split/`).
These contain every ANS test block for the words that each section implements.

There are 153 test blocks placed across 17 section files.
Sections that implement no words with an official ANS test
have no corresponding file.
For example: section 5's inner-interpreter primitives like LIT/BRANCH,
have no directly-callable ANS name, so there are no tests.
Everything else in Core (F.6.1) and the Core Extension subset this
system implements (F.6.2) maps to exactly one section.

Three files were added after that count was made, and sit outside the
one-file-per-assembler-section rule:

1. `27_optional_strings.tests.fs` : the optional String words.
2. `28_double_word.tests.fs` : the Double-Number word set, core and extension, built from the tests at forth-standard.org/standard/double.
3. `29_coreext_strings.tests.fs` : the Core Extension words that no other section file covers: `.(` , `C"` , `S\"` , `CONVERT` , `SAVE-INPUT` , `RESTORE-INPUT` , and the `CORE-EXT` environment query.
4. `30_coreext_misc.tests.fs` : words that had no test elsewhere (`PICK` , `ROLL` , `2R@` , `<>` , `0<>` , `U>` , `WITHIN` , `AGAIN` , `ERASE` , `PARSE` , `SOURCE-ID` , `REFILL` , `TIB` , `#TIB` , `.R` , `U.R`) and the ClaudeForth extensions (`2+` , `BINARY` , `SNAME` , `VHERE` , `V,` , `VC,` , `VALLOT` , `UNUSED` , `VUNUSED`).
5. `31_tools_conditionals.tests.fs` : `[IF]` , `[ELSE]` , `[THEN]` .

Differences from the Published Annex F Suite

Test File Section Independence

The original ANS suite is one continuous stream where later tests reuse
words and constants defined by earlier ones (explicit in the standard's
own F.3 narrative: "these are included in the appropriate test").
Here every section file is independently runnable, in any order, any
number of times, from a single shared harness+prelude load:

- `09_outer_interpreter.tests.fs` (FIND) carries its own copy of `GT1` / `GT2` (`: GT1 123 ; : GT2 ['] GT1 ; IMMEDIATE`) rather than depending on `13_compiling_words.tests.fs` .
- Every section file opens with `MARKER Mnn` (nn = the section

number, e.g. `M08`) right after its header comment, and closes with a bare invocation of `Mnn` as its last line. `MARKER` snapshots `DPHERE` / `CODEHERE` / `VARHERE` / `LATEST` when created and restores all four when invoked (confirmed directly against forth6809.asm's `MARKERW` / `DOMARKER`), so running a section file and then letting it invoke its own marker leaves the dictionary, value-space and search chain exactly as they were right after `ttester.fs` + `00_test_prelude.fs` were loaded — no leftover state for the next section file to trip over, whatever order they run in.
(`BASE` is not part of that snapshot, see *Editing Test Files*.)

- `18_memory.tests.fs`'s `MOVE` test depends on the `FBUF` / `SBUF` / `SEEBUF` state left behind by the immediately preceding `FILL` test — this is fine, since both are in the same file and thus inside the same `MARKER` bracket.
- Several `20_numeric_output.tests.fs` tests use `MAX-BASE`, `#BITS-UD`, and `S=`, all provided by `00_test_prelude.fs`, which sits outside every section's own `MARKER` bracket and is loaded once, up front.

The load order is simply: `ttester.fs`, `00_test_prelude.fs`, then any subset of section files, in
any sequence — see `ans_test_runner.py` (one level up), which automates exactly this against either MAME or a real serial-connected MECB6809.

Deviations from Annex F

Where the section files differ from the published test text:

- **Cross-reference notes.** Every "See F.x.x.xxxx WORD." note and narrative line is a `\` comment, so none is interpreted as Forth.
- **One test per line.** Tests sit on their own lines, never after a `\` comment, apart from the layout-sensitive cases described in *Editing Test Files*.
- **`TRUE` and `FALSE`.** Their tests (`F.6.2.1485 FALSE`, `F.6.2.2298 TRUE`) are at the end of `26_abort_quit_headers.tests.fs`, alongside that section's other ROM-resident, hand-built words.
- **`BASE` handling.** Sections 08, 11, 12, 24 and 26 change `BASE` and restore it explicitly, so no file leaks a base change into the next.
- **16-bit cell (section 24).** The `>IN` test uses `12345` where the original used `123456`, which wraps in a 16-bit cell. The standard itself later made the same change for 16-bit systems. The second `>IN` test keeps the words it deliberately skips (`GCD calculation`).

- **`ENVIRONMENT?` erratum (section 24).** The published `X:deferred` test, `DUP 0= XOR INVERT -> <TRUE>`, can never pass on any system. It checks instead that the answer is a single well-formed flag.
- **`ENVIRONMENT?` query table (section 24).** Annex F tests only `X:deferred` and `X:notfound`. Section 24 also checks every query in the standard table. Word-set queries (`CORE`, `CORE-EXT`, `DOUBLE`, `DOUBLE-EXT`) return a value flag and then the recognized flag, so each expects `<TRUE> <TRUE>`. `/HOLD` and `/PAD` are checked only against the standard minimums (34 and 84).
- **Double-Number (section 28).** The constants `MAX-2INT`, `MIN-2INT`, `HI-2INT` and `LO-2INT` are computed from `MAX-INT`, `MIN-INT` and `D2/` rather than hard-coded, and `dbl1` / `dbl2` use `M*/`, so the file also checks that these words agree with each other. A `MAX-D` query is checked against `MAX-2INT`. The `D.` / `D.R` test only prints, and is judged by eye: the lines it prints must appear in identical pairs. Typos in the source text were corrected (`0D-`, a stray `1s`), and the original constant block was replaced.
- **Core Extension strings (section 29).** Written for this project. A terminal source cannot be rewound across lines, so `SAVE-INPUT` and `RESTORE-INPUT` are tested only for the saved shape and the failure paths. `S\"` and `CONVERT` have tests that cover each escape and the digit scan.
- **Misc words (section 30).** Written for this project, following the Core and Core-Ext suites for the standard words. `.R` and `U.R` are visual: the lines shown must appear in identical pairs.
- **Conditional compilation (section 31).** The standard's own `[IF]` tests plus use inside definitions, nesting, case, look-alike names and the end of an `EVALUATE` string. The last two tests are spread over several lines, so a runner waits for a reply timeout on each of their interior lines.
- **`SM/REM` (section 15).** One expected remainder is corrected to follow the sign-of-dividend pattern of its neighbours.
- **`CATCH` / `THROW` (section 11).** The undefined-word case expects `-13`.
- **Reporting.** Each file starts with `TEST-BEGIN` and ends with `TEST-END` (see *Test Reporting*). `ttester.fs` has no `TEST-REPORT` word.

Defects in `forth6809.asm` that these tests exposed are recorded in `bug_fixes.md`.

Known Limitations and Open Items

- **Compliance is not claimed.** Annex F is a necessary check, not proof of compliance. Many Annex F tests are adapted for a 16-bit cell and the suite excludes the word sets listed below.
- **`ENVIRONMENT?` coverage.** `X:deferred` is not in the query table, so it answers false. The test accepts either answer. `WORDLISTS` is deliberately absent (no Search-Order word set).
- **`RESTORE-INPUT` is only partly tested.** It cannot restore across `REFILL` or into a different source, and the success path needs a multi-line source.
- **`ABORT` is silent.** It prints nothing and returns to the prompt, and `-1 THROW` at the top level also prints nothing. `ABORT"` message behaviour at the top level is not covered by the suite.
- **Layout-sensitive tests** (see *Editing Test Files*) cannot be reflowed, and pasting by hand needs the same care as the runner.
- **Floating-point build option.** `ttester.fs` stays in its original form with the floating branches in place. Only the runner removes them while sending. A manual paste of `ttester.fs` therefore runs slowly (every dead line passes through the target's `[ELSE]` skip).

Tests Not Included

The following tests are not included because the word set is not implemented or is implemented only in part:

1. Facility. Only `KEY?` is implemented. No tests are included for it.
2. File-Access.
3. Floating-Point.
4. Memory-Allocation.
5. Search-Order.

Search-Order's absence is documented explicitly in the ClaudeForth documentation, Section 4.5.

Most of the Programming-Tools word set is also omitted (AHEAD, CS-PICK, CS-ROLL, N>R, [THEN] etc.). They have no ANS test coverage relevant here.

The Tools word set — `.S`, `WORDS`, `DUMP` are non-standard extensions, so they have no official ANS test cases.